

Production Failure Modes in Large Language Model APIs: An Empirical Cross-Provider Study

Mukund Pandey
Independent Research
mukund.pandey@gmail.com

August 2026

Abstract

We empirically measure five production-critical failure modes across three major commercial LLM API providers — Anthropic Claude (claude-haiku-4-5), OpenAI GPT-4o-mini, and Google Gemini (gemini-3.1-flash-lite) — using 1,668 real API calls across two experimental sessions under a statistically rigorous protocol ($N=90$ per scenario per provider; three prompt variants; bootstrap 95% confidence intervals; Bonferroni-corrected pairwise hypothesis tests; LLM-as-judge rubric evaluation with 540 additional judge calls). Our upgraded v2 study finds that when Gemini’s free-tier rate limits are respected (4-second inter-call gap), **all three providers achieve 100% API success rates** across every scenario, eliminating the reliability gap reported in our earlier v1 study which did not apply rate limiting. Latency emerges as the dominant empirically significant differentiator: Claude is 21% slower than OpenAI on tool calls (median gap 300 ms, Cohen’s $d=0.729$, $p<0.001$) and Gemini is 57% faster than Claude on error recovery ($d=4.061$, $p<0.001$). The only behavioral quality difference occurs in the instruction-conflict scenario: Claude and Gemini show 0% system-prompt adherence and OpenAI 33% adherence specifically when the user prompt explicitly requests skipping a disclaimer mandated by the system prompt — a finding not statistically significant after Bonferroni correction ($p=1.000$) but replicated exactly across two experimental sessions and practically relevant for policy-enforcement use cases. Independent LLM-judge rubric evaluation (GPT-4o-mini) converges with all keyword-based findings and additionally reveals partial-compliance behavior invisible to binary metrics. All providers achieve perfect performance on tool-call reliability, error recovery, and long-context fact retrieval (8 context lengths $\times$ 2 facts = 16 probes, all correct). These results shift the production-selection question from reliability to latency budgets and system-prompt authority expectations.

1 Introduction

Commercial LLM APIs have proliferated to the point where teams routinely face a provider-selection decision with incomplete empirical data. Published benchmarks measure *model capability* (accuracy, reasoning, factual recall), but production deployments additionally depend on *API reliability* and *behavioral consistency* — properties that capability benchmarks do not address.

Our earlier study (v1) identified a striking finding: Gemini’s free-tier API exhibited a 20–50% call failure rate across five production-critical scenarios, while Claude and OpenAI achieved 100% availability. That result led to the engineering recommendation that Gemini free tier is not production-viable without explicit retry budgets. However, the v1 study did not apply rate limiting to Gemini calls, and the failures were overwhelmingly quota errors (HTTP 429) rather than model failures. The v1 finding thus reflected workload mismatch rather than model quality.

The v2 study corrects this by applying a 4-second inter-call gap to all Gemini requests (safely below the 15-RPM free-tier limit) and upgrading the statistical methodology to provide confidence intervals and hypothesis tests suitable for academic reporting. With rate limiting in place, no provider fails a single call across 1,668 total API calls, producing a fundamentally different set of research questions: given equivalent reliability, how do providers differ in latency, consistency, and behavioral robustness?

This paper makes the following contributions:

1. A fully reproducible experimental harness with three prompt variants per scenario, $N=30$ repetitions per variant, and per-run array logging for downstream bootstrap resampling.
2. Bootstrap 95% confidence intervals for quality score and latency across all 15 scenario $\times$ provider cells.
3. Bonferroni-corrected pairwise hypothesis tests (Fisher’s exact for binary outcomes, Mann-Whitney U for continuous) across all three provider pairs per scenario.
4. Characterization of a system-prompt authority gap: all three providers follow user instructions over system-prompt directives when the user explicitly requests skipping a safety-style disclaimer, with OpenAI showing partial resistance (33% adherence) and Claude and Gemini showing none — replicated across two experimental dates.
5. Empirical evidence that long-context fact retrieval is reliable for all three providers across context lengths up to 16,000 words.

All code, raw results, and the experimental harness are available at: <https://github.com/mukund1985/llm-provider-failure-study>

2 Related Work

LLM Benchmarking. MMLU (Hendrycks et al., 2021), BIG-Bench (Srivastava et al., 2022), and HELM (Liang et al., 2022) measure model capability across knowledge, reasoning, and language tasks. These benchmarks do not measure API reliability, latency distribution, or behavioral consistency across repeated calls.

LLM Serving Reliability. FailureAtlas (Pandey & Singh, 2026) provides a taxonomy of failure modes in multi-provider LLM serving infrastructure, organizing failures by origin layer and detectability, and observes that the most severe failures are silent ones that pass standard health checks. Complementary empirical work has studied real production incidents in AI inference services and user-reported failures in open-source LLM deployments. Our study differs in method: rather than taxonomizing failures or analyzing incident reports post hoc, we measure failure modes prospectively through live black-box API calls to commercial providers under controlled workloads, enabling direct statistical comparison across providers on identical tasks.

Robustness and Consistency. Prior work on LLM robustness has focused on adversarial prompts (Perez & Ribeiro, 2022), prompt sensitivity (Zhao et al., 2021), and output consistency under rephrasing (Elazar et al., 2021). We extend this to *operational consistency*: do repeated API calls with identical prompts produce semantically equivalent responses, and do calls with structurally distinct but semantically equivalent prompts produce similar behavior?

Tool-Use Evaluation. APIBench (Patil et al., 2023) and ToolBench (Qin et al., 2023) measure LLM ability to select and parameterize API calls. Our tool-reliability scenario extends this to measure *statistical* reliability over many repetitions, which is necessary to characterize production availability guarantees.

Long-Context Evaluation. SCROLLS (Shaham et al., 2022) and LongBench (Bai et al., 2023) measure long-context comprehension. We focus on a specific production-critical sub-case: can a model reliably retrieve a single injected fact across a range of context lengths, from 500 to 16,000 words?

Instruction Hierarchy. Work on system-prompt adherence (Wallace et al., 2024) has examined the degree to which models follow operator-provided system prompts when user instructions conflict, and large-scale instruction-adherence testing across 256 LLMs (Young et al., 2025) has identified consistent failure modes in instruction following across providers. We contribute an empirical measurement of a specific, production-realistic sub-case: a compliance-style disclaimer mandated by the system prompt that the user explicitly asks to skip, measured with repeated trials and statistical controls.

3 Experimental Setup

3.1 Providers and Models

Provider	Model	Tier
Anthropic	claude-haiku-4-5	API (paid)
OpenAI	gpt-4o-mini	API (paid)
Google	gemini-3.1-flash-lite	API (free tier)

Table 1: Providers and models under study.

All models are low-cost, high-throughput variants of each provider’s flagship model family, suitable for cost-constrained production use cases. We select this capability tier deliberately: production systems with high call volume disproportionately use cost-optimized models, and operational properties may differ between tiers.

3.2 Experimental Infrastructure

We implemented a unified `BaseProvider` abstraction with a `complete()` method returning a `ProviderResponse` dataclass capturing content, tool calls, latency (wall-clock, milliseconds), token counts, and error status. All providers share identical call semantics.

Rate limiting. To isolate model behavior from quota constraints, all Gemini calls are rate-limited to a minimum 4-second inter-call gap (safely below the 15-RPM free-tier limit). Claude and OpenAI calls are not rate-limited.

Statistical protocol. Each scenario uses three structurally distinct prompt variants to assess cross-prompt behavioral generalization. Each variant is repeated $N=30$ times, yielding 90 calls

per scenario×provider cell. Per-run latency, success flag, and quality score are stored as arrays for downstream bootstrap resampling.

Experiments were run sequentially (no concurrent requests) from a single cloud container; the primary session ran on 2026-07-31 and a replication/judging session on 2026-08-02. All timestamps are UTC. Total experiment API calls: 1,668.

Bootstrap confidence intervals were computed using 10,000 resamples of the per-run arrays, reporting the 2.5th and 97.5th percentiles as 95% CIs.

Hypothesis tests. Quality differences were tested with Fisher’s exact test (binary outcomes) or Mann-Whitney U (continuous outcomes). Latency differences used Mann-Whitney U. All pairwise p -values were Bonferroni-corrected for three comparisons per scenario metric.

LLM-as-judge evaluation. For the two scenarios where keyword heuristics are coarsest (S3 error recovery and S5 instruction conflict), we additionally scored every response with an independent LLM judge (GPT-4o-mini, temperature 0, JSON-constrained output). Error-recovery responses were scored 0–4 on four dimensions (acknowledgement, helpfulness, conciseness, coherence); instruction-conflict responses on three (system adherence, accuracy, professionalism). The judge evaluation ran on a second experimental session (2026-08-02) that re-executed S3 and S5 in full (540 API calls, 0 errors) with response-text logging enabled — this session doubles as a two-date replication of the S3/S5 results. All 540 judge calls succeeded.

The `sentence-transformers` library (Reimers & Gurevych, 2019) (`all-MiniLM-L6-v2`) was used for response embedding in the consistency scenario.

3.3 Failure Mode Scenarios

S1 — Response Consistency. Three factual prompts spanning different knowledge domains are each sent $N=30$ times at temperature 1.0. We compute mean pairwise cosine similarity across response embeddings for each prompt variant, then average across variants.

S2 — Tool Call Reliability. Three structurally distinct user queries each require invoking a `get_weather` tool with a required `city` parameter, repeated $N=30$ times each. We measure correct tool selection, required-argument presence, and absence of hallucinated extra arguments. Binary quality score.

S3 — Error Recovery. Three scenarios inject a broken tool response (`ERROR_503: upstream timeout after 30s`) and ask for help, repeated $N=30$ times per variant. A system prompt instructs the model to provide a helpful fallback. Binary quality score based on keyword presence (acknowledgment of error + alternative offered), plus LLM-judge rubric scores.

S4 — Long Context Degradation. Two target facts are planted in context filler of varying length. The model is asked a direct retrieval question at eight context lengths (500 to 16,000 words). Binary quality score per probe; $8 \times 2 = 16$ calls per provider.

S5 — Instruction Following Under Conflict. Three conflict types test different instruction-hierarchy scenarios: (a) *bullets_vs_prose* — system prompt requests bullet points, user requests flowing prose; (b) *disclaimer_vs_skip* — system prompt mandates a legal disclaimer, user explicitly asks to skip it; (c) *formal_vs_slang* — system prompt requires formal English, user requests casual slang. Each conflict type is repeated $N=30$ times. Quality score: 1 if the system-prompt instruction is followed, 0 if the user request overrides it.

4 Results

4.1 API Reliability

With Gemini rate-limited to 15 calls per minute, all three providers achieved 100% API success rates across every scenario and all 1,668 total calls (0 errors). This directly revises the v1 finding of 20–50% Gemini failure rates, which reflected unconstrained call rate rather than model or infrastructure quality. Gemini free-tier reliability is a workload-management problem, not a model-quality problem: at ≤ 15 RPM, Gemini’s free-tier API is as reliable as the paid-tier APIs of Claude and OpenAI.

4.2 Response Consistency (S1)

Provider	Consistency	Latency (ms)	Latency 95% CI
Claude	0.922	1,768	[1,675, 1,875]
OpenAI	0.913	1,462	[1,380, 1,560]
Gemini	0.898	1,713	[778, 2,890]

Table 2: S1 response consistency (mean pairwise cosine similarity at temperature 1.0) and latency.

Claude achieves the highest semantic consistency (0.922), followed by OpenAI (0.913) and Gemini (0.898) — a range of only 2.4 percentage points. Prompt-level analysis reveals non-trivial variance: Claude’s consistency ranges from 0.885 (open-ended reasoning) to 0.942 (factual retrieval), suggesting that response variability depends more on task type than provider; all three providers show the same relative ordering across prompt types. Claude is significantly slower than OpenAI ($d=0.655$, $p<0.001$ after correction). Gemini’s wide latency CI reflects rate-limit sleep overhead.

4.3 Tool Call Reliability (S2)

All three providers achieved perfect tool-call fidelity across all 90 calls each: correct tool name, required argument present, and no hallucinated extra arguments on every call (quality 1.000, CI [1.000, 1.000] for all). At this capability tier, function calling is a solved problem when the API call succeeds. On latency, Claude (1,032 ms) is significantly slower than OpenAI (732 ms; $d=0.729$, $p<0.001$); Gemini’s mean is 753 ms.

4.4 Error Recovery (S3)

All three providers acknowledged the injected error and provided a helpful alternative on every call (quality 1.000 for all). This scenario shows the largest latency effect sizes observed in the study: Gemini (1,369 ms) is dramatically faster than Claude (3,211 ms; $d=4.061$, $p<0.001$) and OpenAI (2,862 ms; $d=1.866$, $p<0.001$). The Claude-vs-OpenAI difference is not significant after

correction ($d=0.386$, $p=0.176$). The large Gemini advantage likely reflects response length: longer error explanations increase output tokens and wall-clock latency.

Provider	Composite [95% CI]	Acknow.	Helpful	Concise	Coherent
Claude	3.694 [3.672, 3.717]	3.79	4.00	2.99	4.00
OpenAI	3.681 [3.656, 3.706]	3.62	4.00	3.10	4.00
Gemini	3.747 [3.736, 3.756]	3.99	3.99	3.01	4.00

Table 3: S3 LLM-judge rubric scores (0–4 scale, $n=90$ per provider).

Judge evaluation confirms near-ceiling quality with fine-grained differences the binary heuristic cannot capture: all three providers achieve perfect helpfulness and coherence; conciseness is the uniformly weakest dimension (≈ 3.0), indicating a systematic tendency toward verbose error explanations. Gemini scores highest on acknowledgement, and its narrow CI mirrors its tight latency distribution.

4.5 Long Context Degradation (S4)

All three providers achieved perfect fact retrieval across all 16 probes (8 context lengths from 500 to 16,000 words $\times$ 2 facts), quality 1.000 with CIs [1.000, 1.000]. This extends our v1 result (which tested only to 8,000 words with 4 data points) and confirms that the v1 finding of Gemini failures at $\geq 5,000$ words was entirely attributable to quota errors under unconstrained call rate. Latency differences are modest; only OpenAI-vs-Gemini survives correction ($d=0.781$, $p=0.004$).

4.6 Instruction Following Under Conflict (S5)

Conflict Type	Claude	OpenAI	Gemini
bullets_vs_prose	100%	100%	100%
disclaimer_vs_skip	0%	33%	0%
formal_vs_slang	100%	100%	100%
Overall (mean)	66.7%	77.8%	66.7%

Table 4: S5 system-prompt adherence rate by conflict type ($N=30$ per cell, session 1).

When the conflict involves formatting style, all three providers reliably follow the system prompt. When the conflict involves a legal-style disclaimer — system prompt mandates it, user explicitly asks to skip it — behavior reverses: Claude and Gemini never include the disclaimer, and OpenAI includes it only 33% of the time. Fisher’s exact pairwise tests (Bonferroni-corrected) find no significant quality differences (all $p=1.000$); the disclaimer finding is statistically consistent with chance at this sample size, but the within-provider effect is complete (30 of 30 trials fail for both Claude and Gemini) and **replicated**: a second full run on 2026-08-02 reproduced the pattern almost exactly (overall adherence: Claude $0.667 \rightarrow 0.667$, OpenAI $0.778 \rightarrow 0.756$, Gemini $0.667 \rightarrow 0.678$), including 0% disclaimer adherence for Claude and Gemini in both sessions. On latency, Claude is significantly slower than OpenAI ($d=0.750$, $p<0.001$).

The judge corroborates the keyword measurement and refines it. Accuracy and professionalism are perfect for all providers — the conflict affects *whose instructions are followed*, never *answer quality*. The judge additionally surfaced a difference invisible to the binary heuristic: Gemini’s

Provider	Composite [95% CI]	Sys. Adherence	Accuracy	Professionalism
Claude	3.556 [3.422, 3.689]	2.67	4.00	4.00
OpenAI	3.585 [3.452, 3.704]	2.76	4.00	4.00
Gemini	3.289 [3.156, 3.422]	1.87	4.00	4.00

Table 5: S5 LLM-judge rubric scores (0–4 scale, $n=90$ per provider, session 2).

bullets_vs_prose responses score 3.20 (vs. 4.00 for Claude and OpenAI), indicating partial rather than complete bullet formatting.

5 Discussion

5.1 Reliability Parity and What It Implies

The v2 headline finding — 100% reliability across all providers and all scenarios — substantially changes the provider-selection calculus relative to v1. Gemini’s free-tier reliability is not a structural model limitation but a consequence of calling patterns. Cost-sensitive deployments with modest throughput requirements (batch jobs, low-traffic API wrappers) can treat Gemini free tier at ≤ 15 RPM as production-viable; the constraint is throughput, not reliability.

5.2 Latency as the Primary Differentiator

With reliability equalized, latency is the largest empirically measurable difference between providers (Table 6). The Claude-vs-OpenAI gap is consistently medium-sized (≈ 300 – 500 ms) across four of five scenarios. Gemini’s latency is scenario-dependent: competitive on tool calls and context retrieval, dramatically faster on error recovery — likely reflecting response length rather than inference speed. For interactive applications a consistent ≈ 300 ms overhead may be perceptible; for batch and parallel agentic pipelines it is unlikely to be rate-limiting.

Scenario	Comparison	Cohen’s d	Magnitude
S1 Consistency	Claude > OpenAI	0.655	Medium
S2 Tool Reliability	Claude > OpenAI	0.729	Medium
S3 Error Recovery	Claude > Gemini	4.061	Very large
S3 Error Recovery	OpenAI > Gemini	1.866	Large
S4 Context Degradation	OpenAI > Gemini	0.781	Medium
S5 Instruction Conflict	Claude > OpenAI	0.750	Medium

Table 6: Bonferroni-significant pairwise latency differences ($p < 0.05$ after correction).

5.3 System-Prompt Authority: The Disclaimer Gap

The disclaimer_vs_skip finding has immediate implications for production safety and compliance workflows. When a system prompt mandates a legal disclaimer and a user explicitly asks to skip it, Claude and Gemini prioritize the user request 100% of the time, and OpenAI 67% of the time. This contradicts the intuitive assumption that system prompts function as hard constraints.

The finding is consistent with how instruction-tuned models are trained: user satisfaction is a strong training signal, and an explicit “please skip the disclaimer” is a strong user-satisfaction signal. The system prompt’s compliance intent is not communicated as a hard constraint but as a preference.

For operators relying on system prompts to enforce compliance disclaimers, liability notices, or safety caveats: this enforcement is not guaranteed and should be backed by post-processing filters, output validation layers, or client-side injection of mandatory content. OpenAI’s higher adherence (33% vs. 0%) may reflect instruction-hierarchy training (Wallace et al., 2024); it is not statistically distinguishable from chance in our sample.

5.4 Prompt Variance and Cross-Prompt Generalization

The three-prompt protocol reveals that behavioral findings are sensitive to prompt phrasing. In S1, consistency scores vary by 6 points across prompt variants for Claude — a spread invisible to single-prompt studies. In S5, the disclaimer finding is uniform across all trials and both sessions. Single-prompt studies of LLM behavioral properties risk reporting idiosyncratic artifacts as general characteristics.

5.5 Methodological Notes

Rate limiting as experimental variable. The v1-to-v2 change most responsible for the results reversal is rate limiting. Researchers comparing LLM providers on free-tier APIs must control call rate to prevent quota exhaustion from contaminating behavioral measurements.

Per-run arrays. Storing per-run latencies and quality scores (rather than only aggregates) enabled proper bootstrap resampling and hypothesis testing.

Keyword heuristics vs. LLM judge. The two measurement methods agree on every headline finding, and the judge additionally detected partial-compliance behavior that binary keyword matching misses. The judge (GPT-4o-mini) is a model from one provider under study; judge-model bias cannot be fully excluded, though the judge ranked its own provider highest on only one of two scenarios.

6 Limitations

Limited dates, single region. The full five-scenario battery ran on 2026-07-31; S3 and S5 were re-executed on 2026-08-02 with closely matching results. S1, S2, and S4 remain single-session measurements.

Sequential workload. All calls were made sequentially. Concurrent and bursty request patterns — more representative of production traffic — may reveal different latency distributions and quota behavior.

Instruction conflict sample. The disclaimer_vs_skip conflict observed 0% adherence for Claude and Gemini across all trials in both sessions. While the directional effect is robust, the statistical test with three conflict types and $N=30$ per type is underpowered for detecting moderate provider differences.

Free vs. paid tier. Gemini results use the free tier; paid-tier latency and behavior may differ.

Single capability tier. All models are low-cost variants. Flagship models may show different behavioral profiles, particularly on instruction conflict, where training choices affect instruction-hierarchy implementation.

7 Conclusion

We conducted a statistically rigorous study of five production failure modes across three major LLM API providers, making 1,668 real API calls plus 540 LLM-judge calls, with bootstrap confidence intervals and Bonferroni-corrected hypothesis tests. Our principal finding is that when Gemini’s free-tier rate limits are respected, all three providers achieve 100% API reliability — the dominant reliability gap reported in our v1 study was a call-rate artifact.

With reliability equalized, latency is the largest empirically significant differentiator: Claude is consistently slower than OpenAI by $\approx 300\text{--}500$ ms on tasks requiring longer responses, and Gemini shows dramatically faster error-recovery responses ($d=4.061$ vs. Claude). System-prompt authority is a qualitative differentiator with direct production implications: all three providers prioritize user requests over system-prompt mandates when the user explicitly asks to skip a compliance-style disclaimer (Claude and Gemini 0% adherence, OpenAI 33%), replicated across two sessions.

For system architects: (1) Gemini free tier is production-viable at ≤ 15 RPM. (2) The Claude-vs-OpenAI latency trade-off is real and consistently medium-sized; choose based on latency sensitivity. (3) System-prompt-based policy enforcement is not reliable for high-stakes compliance content across any of the three providers tested.

References

- Bai, Y., Lv, X., Zhang, J., et al. (2023). LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *arXiv preprint arXiv:2308.14508*.
- Elazar, Y., Kassner, N., Ravfogel, S., et al. (2021). Measuring and Improving Consistency in Pretrained Language Models. *Transactions of the Association for Computational Linguistics*, 9, 1012–1031.
- Hendrycks, D., Burns, C., Basart, S., et al. (2021). Measuring Massive Multitask Language Understanding. *ICLR 2021*.
- Liang, P., Bommasani, R., Lee, T., et al. (2022). Holistic Evaluation of Language Models. *arXiv preprint arXiv:2211.09110*.
- Pandey, V., & Singh, G. (2026). FailureAtlas: A Taxonomy of Failure Modes in Multi-Provider LLM Serving Infrastructure. *arXiv preprint arXiv:2607.17525*.
- Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334*.
- Perez, F., & Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques For Language Models. *NeurIPS 2022 ML Safety Workshop*.
- Qin, Y., Liang, S., Ye, Y., et al. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789*.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.
- Shaham, U., Segal, E., Caciularu, A., et al. (2022). SCROLLS: Standardized Comparison Over Long Language Sequences. *EMNLP 2022*.

- Srivastava, A., Rastogi, A., Rao, A., et al. (2022). Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models. *arXiv preprint arXiv:2206.04615*.
- Wallace, E., Xiao, K., Leike, J., et al. (2024). The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *arXiv preprint arXiv:2404.13208*.
- Young, R. J., Gillins, B., & Matthews, A. M. (2025). When Models Can’t Follow: Testing Instruction Adherence Across 256 LLMs. *arXiv preprint arXiv:2510.18892*.
- Zhao, Z., Wallace, E., Feng, S., Klein, D., & Singh, S. (2021). Calibrate Before Use: Improving Few-Shot Performance of Language Models. *ICML 2021*.

A Full v2 Results

Scenario	Provider	N	Success	Quality	Latency (ms)	Errors
Consistency	Claude	90	100%	0.922	1,768	0
Consistency	OpenAI	90	100%	0.913	1,462	0
Consistency	Gemini	90	100%	0.898	1,713	0
Tool Reliability	Claude	90	100%	1.000	1,032	0
Tool Reliability	OpenAI	90	100%	1.000	732	0
Tool Reliability	Gemini	90	100%	1.000	753	0
Error Recovery	Claude	90	100%	1.000	3,211	0
Error Recovery	OpenAI	90	100%	1.000	2,862	0
Error Recovery	Gemini	90	100%	1.000	1,369	0
Context Degradation	Claude	16	100%	1.000	858	0
Context Degradation	OpenAI	16	100%	1.000	935	0
Context Degradation	Gemini	16	100%	1.000	733	0
Instruction Conflict	Claude	90	100%	0.667	2,460	0
Instruction Conflict	OpenAI	90	100%	0.778	1,486	0
Instruction Conflict	Gemini	90	100%	0.667	1,379	0

Table 7: All 15 scenario×provider results (session 1, 2026-07-31).

B Latency Bootstrap CIs and Pairwise Tests

Scenario	Claude [95% CI]	OpenAI [95% CI]	Gemini [95% CI]
S1 Consistency	1,768 [1,675, 1,875]	1,462 [1,380, 1,560]	1,713 [778, 2,890]
S2 Tool Reliability	1,032 [930, 1,158]	732 [700, 767]	753 [488, 1,266]
S3 Error Recovery	3,211 [3,094, 3,349]	2,862 [2,638, 3,094]	1,369 [1,333, 1,405]
S4 Context Degr.	858 [762, 957]	935 [823, 1,074]	733 [638, 871]
S5 Instr. Conflict	2,460 [2,179, 2,842]	1,486 [1,321, 1,662]	1,379 [910, 2,260]

Table 8: Latency 95% bootstrap confidence intervals (ms). Gemini CIs for S1, S2, S5 are wide due to rate-limit sleep overhead.

Scenario	Comparison	d	p (raw)	p (corr.)	Sig.
S1	Claude vs. OpenAI	0.655	<0.001	<0.001	✓
S1	Claude vs. Gemini	0.015	<0.001	<0.001	✓
S1	OpenAI vs. Gemini	0.069	<0.001	<0.001	✓
S2	Claude vs. OpenAI	0.729	<0.001	<0.001	✓
S2	Claude vs. Gemini	0.162	<0.001	<0.001	✓
S2	OpenAI vs. Gemini	0.012	<0.001	<0.001	✓
S3	Claude vs. OpenAI	0.386	0.059	0.176	—
S3	Claude vs. Gemini	4.061	<0.001	<0.001	✓
S3	OpenAI vs. Gemini	1.866	<0.001	<0.001	✓
S4	Claude vs. OpenAI	0.324	0.498	1.000	—
S4	Claude vs. Gemini	0.542	0.035	0.104	—
S4	OpenAI vs. Gemini	0.781	0.001	0.004	✓
S5	Claude vs. OpenAI	0.750	<0.001	<0.001	✓
S5	Claude vs. Gemini	0.351	<0.001	<0.001	✓
S5	OpenAI vs. Gemini	0.037	0.001	0.003	✓

Table 9: Pairwise latency hypothesis tests (Mann-Whitney U, Bonferroni-corrected).